MISB MOTION IMAGERY STANDARDS BOARD	MISB ST 1901.3
STANDARD	
Motion Imagery Metadata (MIMD): Modeling Rules	**2025-06-11**

1 Scope

The Motion Imagery Metadata (MIMD) documents consist of the MIMD Modeling Rules, MIMD Model, and Model-to-KLV Transmutation Instructions. This standard defines the MIMD Modeling Rules. The modeling rules are a subset of standardized UML class diagrams [1] specifically minimized to enable transforming, or transmuting, class instances to KLV. Other formats such as XML or JSON may have transmutation instructions developed for them in future revisions. MISB ST 1903 [2] defines the MIMD Model, which is a UML model organized as a hierarchy of information including metadata for temporal, platform, payload, sensor, command, automated processes, exploitation, security, and more. MISB ST 1902 [3] defines the Model-to-KLV Transmutation Instructions for constructing bandwidth efficient KLV structures from the model. This document, in concert with MISB ST 1902, MISB ST 1903 and its supporting MIMD model detail documents (MISB ST 1904 [4], MISB ST 1905 [5], MISB ST 1906 [6], MISB ST 1907 [7], and MISB ST 1908 [8]), provides a suite of standards embodying the MIMD architecture.

2 References

[1] OMG, *UML-v2.5.1 - Unified Modeling Language,* 2017-12.

[2] MISB, *ST 1903.3 - Motion Imagery Metadata (MIMD): Model,* 2025-06-11.

[3] MISB, *ST 1902.3 - Motion Imagery Metadata (MIMD): Model-to-KLV Transmutation Instructions,* 2025-06-11.

[4] MISB, *ST 1904.3 - Motion Imagery Metadata (MIMD): Base Attributes,* 2025-06-11.

[5] MISB, *ST 1905.3 - Motion Imagery Metadata (MIMD): Platform,* 2025-06-11.

[6] MISB, *ST 1906.3 - Motion Imagery Metadata (MIMD): Staging System,* 2025-06-11.

[7] MISB, *ST 1907.3 - Motion Imagery Metadata (MIMD): Payload,* 2025-06-11.

[8] MISB, *ST 1908.3 - Motion Imagery Metadata (MIMD): Imager System,* 2025-06-11.

[9] MISB, *MISB ST 0107.5 - KLV Metadata in Motion Imagery,* 2021-10-21.

[10] MISB, *MISP-2025.1: Motion Imagery Handbook,* 2025-06-11.

[11] ISO/IEC, *10646:2020 - Information Technology - Universal Coded Character Set (UCS).*

[12] IEEE, *754-2019 - Standard for Floating-Point Arithmetic.*

[13] NIST, *Special Publication 330 - The International System of Units (SI), 2019 Edition.* (https://doi.org/10.6028/NIST.SP.330-2019)

3 Revision History

Version	Approved Date	Change Description
ST 1901.3	2025-06-11	• Clarified UML class diagrams are informative and textual class definitions (tables) are normative • Clarified use and invocation of ST0107; added requirement -09 • Changed method for producers/receivers to determine the list organization method; added requirements -10, -11, -12, -13 & -144 (change is not backwards compatible with version ST 1901.2) • Added clarification when using inheritance relationships; children instances cannot change specializations; added requirements -15 and -16 • Added discussion on the hierarchy of classes and instances, defining "class ownership" and "paths" in the hierarchy; added requirement -17 • Made editorial changes to requirements -01, -02, -04 & -06, • Editorial changes

4 Definitions of Terms, Acronyms, and Initialisms

4.1 Terms

The following lists the terms and their definitions this document uses.

Fixed-Full-List	The Full-List a producer uses for a Fixed Order list; see Section 7.2.1.1.1.1
Fixed-Packet-List	The Fixed Order list of elements a producer puts into a packet; a subset of the Fixed-Full-List; see Section 7.2.1.1.1.1
Full-List	A producers "in-memory" list of class instance; see Section 7.2.1.1.1
MIMD Model	A set of hierarchical UML class diagrams (informative) and textual class definitions (normative) based on the rules defined in Section 7 and Section 8
Packet-List	The list of class instance elements a producer includes in a packet; a subset of the Full-List; see Section 7.2.1.1.1
Report-on-Change	A method to reduce bandwidth over time by only writing changed data in a data packet instead of writing pre-stated values in every packet; see Motion Imagery Handbook
ROC_Unknown	A Report-on-Change status indicating the state of an attribute is not known; see Motion Imagery Handbook

4.2 Acronyms and Initialisms

The following lists the acronyms and initialisms this document uses.

ISR	Intelligence, Surveillance, and Reconnaissance
KLV	Key Length Value
MIMD	Motion Imagery Metadata
MISB	Motion Imagery Standards Board

MISP	Motion Imagery Standards Profile
OMG	Object Management Group
UML	Unified Modeling Language
XML	eXtensible Markup Language

5 Introduction

Motion Imagery metadata facilitates managing, discovering, and providing contextual information for exploiting Motion Imagery. To date the MISB has published numerous metadata standards to address the needs of the ISR Motion Imagery community. New applications demand additions to these standards as well as the creation of new ones, which warrants the MISB to apply a more holistic approach to metadata in general. The purpose of the MIMD suite of standards is to provide a framework that consolidates and organizes information into a singular, unified metadata model, thereby serving many use cases. This MIMD model then transmutes into different formats, such as KLV and XML.

The MIMD standards address different aspects of the MIMD system, which defines the processes for generating efficient KLV (and other future formats) from a UML model. Figure 1 identifies the five components of the MIMD system: MIMD Modeling Rules, MIMD Model, MIMD Instance, MIMD Transmutation Instructions, and MIMD Formats. The blue items represent MISB standards, and the green items represent implementation data.

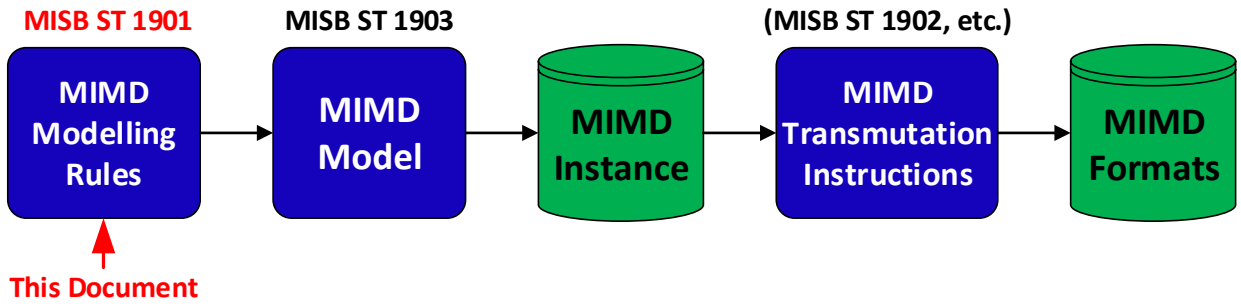


Figure 1: MIMD System Components

The MIMD Modeling Rules comprise a subset of the UML rules for building the MIMD Model. These rules enable transforming, or transmuting, MIMD Instances into MIMD Formats by the MIMD Transmutation Instruction documents.

The MIMD Model is a UML data model following the MIMD Modeling Rules. The Model contains classes, class attributes and defines relationships between classes. MISB ST 1903 and its supporting documents define the MIMD Model.

A MIMD Instance is a data structure implementation of the MIMD Model. While a class defines the allowed contents and rules, an instance is a specific implementation of the class. In other words, a class is the blueprint; the instance is the actual data.

The MIMD KLV Transmutation Instructions provides the “algorithm” for converting a MIMD Model Instance into MIMD Formats. Currently, the only MIMD Format is KLV. MISB ST 1902 defines the MIMD Model-to-KLV Transmutation Instructions that maps instances and their

relationships to hierarchal KLV structures (e.g., Local Sets and Packs). Additionally, these instructions define methods for the "special" model functions for KLV-unique encodings (e.g., temporal compression). The MISB intends to develop other transmutation instructions for other formats such as XML and JSON as needed.

The MIMD Formats are the final product of the MIMD system. The MIMD KLV Format is a bit efficient implementation of the MIMD Instance data. Systems generate custom subsets of the MIMD KLV by selecting appropriate classes and class attributes for their system and following the MIMD Model-to-KLV Transmutation Instructions and requirements. Systems that create metadata and transmute it to MIMD Formats are producers. Systems that decode MIMD Formats and utilize the metadata are receivers.

6 MIMD Model Overview

The MIMD Model is a set of hierarchical UML class diagrams (informative) and textual class definitions (normative) based on the rules defined in Section 7 and Section 8. Section 7 defines the subset of the OMG UML class specification the MIMD Model uses. Section 8 defines extensions beyond the OMG specification including information the MIMD model needs for different encodings (e.g., KLV or XML) and clear definitions for classes along with their attributes.

The MIMD Model defines the allowed hierarchical structure and relationships of classes systems may use to report their metadata. At run-time, systems create instances of the classes they need from the MIMD Model definition; systems therefore will, in general, use a subset of the full MIMD Model.

Producers report metadata for discrete [system defined] time by "transmitting" or "sending" MIMD packets to receivers. The hierarchy of class instances for a given time defines the content of a MIMD packet. Each MIMD packet contains a single MIMD class instance ("root") which includes hierarchical children class instances. The MIMD class instance is for an instant in time, but individual child class instances may include temporal offsets (either positive or negative) to fine tune the timing for all the instance's attribute measurements. The temporal offset applies to all the child class attribute measures in the same manner. For example, a class instance with two attributes (waterTemp and seaState) and a temporal offset of +7 nanoseconds means the measurements for both waterTemp and seaState were 7 nanoseconds after the class's timestamp.

A sequence of MIMD packets forms a MIMD Stream. Producers have a choice to either create full MIMD packets or, to conserve bandwidth, use the Report-on-Change method (see Section 6.1). Full MIMD packets contain all class instances and their attributes an application uses in every packet. Receivers always assume the Report-on-Change technique is in use, so in some situations, producers using full MIMD packets may need to include some Report-on-Change signaling.

6.1 Report-on-Change

The MIMD Model invokes requirements in MISB ST 0107 [9] for Report-on-Change. The Motion Imagery Handbook [10] fully describes the concepts of the Report-on-Change method; this section is a summary for MIMD. With the Report-on-Change method, MIMD producers

optionally report only changes from MIMD packet to MIMD packet, thus eliminating redundant information over time. Receivers always assume producers are using Report-on-Change MIMD packets. Even though producers may choose not to use Report-on-Change, producers need to understand the Report-on-Change rules to know the impact of the receiver's data interpretation when a producer stops reporting data.

Requirement	
ST 1901.3-09	Producers shall adhere to the MISB ST 0107 Report-on-Change requirements.

The Report-on-Change methodology prescribes rules for data expiration and control. The Report-on-Change includes an "Unknown" data state for undefined and expired data items. For clarity, the MIMD documents use the term "ROC_Unknown" for the "Unknown" state name. When a producer is updating a class attribute, the complete heritage of parents (up to the MIMD root instance) are a part of the Report-On-Change packet.

Figure 2 illustrates a series of MIMD packets occurring over time comprising a MIMD Stream. In this example, each packet has a different MIMD instance structure. Each packet in the stream provides metadata for a given time. The colored squares represent different class instances (e.g., red is platform, green is environmental, brown is sensor). In this example, receivers use the Report-on-Change concept to infer the same constant environmental instance (green) in Packets 2, 3, and 4 from Packet 1. Packet 5 shows a new class instance (in blue) un-reported in the stream prior. Systems may also include or exclude attributes within each class instance, as necessary.

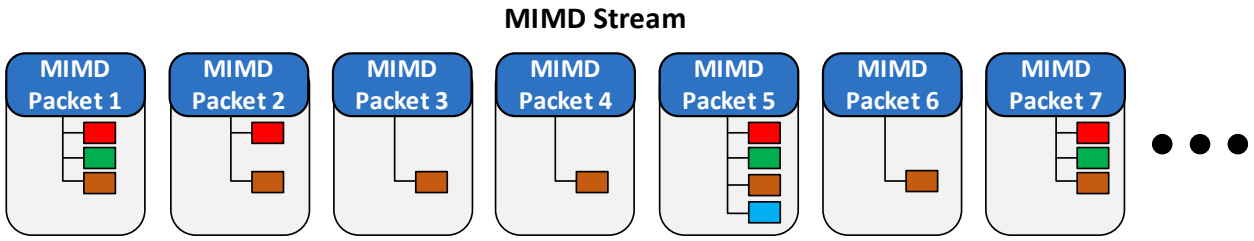


Figure 2: MIMD Stream comprised of MIMD Packets

Report-on-Change extends to attributes within a class. For example, each packet in Figure 2 shows sensor metadata (brown), but the sensor's altitude may be static for all seven packets, so the producer only specifies it in the first packet and omits it in the remaining six packets. Readers would read the first packet and infer, via Report-on-Change rules, the value in packets two through seven.

7 MIMD UML Modeling Rules

The MIMD UML Modeling Rules are a subset of the UML class diagram methodology. The MIMD Model diagram is a hierarchy of **MIMD Model Classes** connected using **MIMD Model Relationships**.

7.1 MIMD Model Classes

MIMD Model Classes are UML classes with a name and one or more attributes. A class is a

collection of information about a given object (a thing, e.g., Sensor) or topic (a concept, e.g., Counting People). For example, a Sensor class contains information about a sensor, such as its location, the number of rows/columns in its sensor array, etc. The class name reflects the collective meaning of the attributes within the class. The class name is a capitalized single word or upper-camel case phrase without spaces (e.g., CamelCase). The naming rules allow the class to map readily and consistently into software.

MIMD Model classes do not include functions. The MIMD Model defines only attributes; the data has no functions or transformations during transmission from the source to destination.

Figure 3 shows an example of a class with the name Platform with three attributes, each with their defined data type.

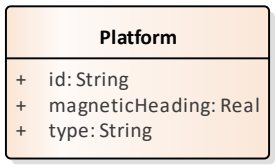


Figure 3: Class Example

7.1.1 Class Attributes

Class attributes are the informational items defining the class content. An attribute's visibility (public, private, etc.) does not apply in the MIMD Model; all attributes are public. Class attributes are not order-specific within the class, and they are in use only one time (i.e., one attribute value per instance per class). Unless otherwise stated, all classes include a set of MIMD Base Attributes and relationships, which are not visible in each class diagram. MISB ST 1904 [4] provides details of the MIMD Base Attributes. MIMD UML diagrams require defining two attribute traits: **Name** and **Type**.

The attribute trait **Name** is a single word or phrase starting with a lowercase letter and in lower-camel case (e.g., attributeName) for multiple words (i.e., no spaces in the names). The Name may not start with a numeric digit but may include digits after the first letter of the name. Name is unique within the scope of the class, which includes the Base Attributes. Attribute Name equivalence is independent of the case of the name, e.g., "engineTemp" and "engineTEMP" are the same name and not unique. Two *different* classes define two different class-scopes; therefore, each class may have one or more attributes using the same Name as the other class. The naming rules allow the attributes to map easily and consistently into software.

The attribute trait **Type** defines the data representation of the attribute value once the class has been instantiated. There are three categories of Type: *Singular Type*, *Enumeration Type*, and *Array Type.* Additionally, a *Tuple Type* defines a sequence of Unsigned integers (for various MIMD internal uses, e.g., instance's "address" for Directed Association) and a type value of *RESERVED* indicates the potential future use of the attribute identifier defined in Section 8.1.1.

7.1.1.1 Singular Type

The *Singular Type* is the basic data type for defining a single value, for example an integer or decimal number. Ideally these types are logical types for modeling and do not have limitations (e.g., value range, precision) that software languages and physical formats impose. This enables modeling to be independent of hardware, operating system, and software limitations. The type's value range may (through its attribute definition) specify minimum and maximum traits, which impose constraints. A MIMD Singular Type name is in upper-camel case (i.e., the first character must be an uppercase letter). Table 1 lists the allowed MIMD class attribute *Singular Types*. MIMD Transmutation Instructions will specify a type's format for use within the target MIMD

Format (e.g., UTF-8 character set from ISO/IEC 10646 [11] for Strings or IEEE 754 floating point value [12] for Reals.)

Table 1: MIMD Class Attribute Singular Types

Type	Definition	Example
String	A sequence of characters supporting multiple language character sets	Model 3
Real	A decimal number (i.e., all rational and irrational numbers)	3.14159
Integer	A signed integer value (e.g., all negative and positive values including zero)	-100
UInt[1]	An unsigned integer value (e.g., all positive values including zero)	127
Boolean	A representation supporting only a true or false values (e.g., 0=false, 1=true)	1

7.1.1.2 Enumeration Type

The *Enumeration Type* is a finite list of enumeration items, one of which assigns to an attribute's value. An enumeration item has three parts: **Enumeration Identifier**, **Name**, and **Definition**. The **Enumeration Identifier** is a unique (within the enumeration list) unsigned integer to identify one of the enumeration items in the list, numerically. The **Name** is a string, unique within the scope of the enumeration list, which identifies one of the enumerations items in the list. The Name does not include spaces but may use underscores ("_") as a word delimiter. The **Definition** describes the meaning of the enumeration or references a document section with further information.

MIMD Model diagrams represent Enumeration Types as an object with a UML stereotype of "≪enumerations≫" and green color shading. One or more classes may link to the enumeration with *undirected* Association. The enumeration object may include the enumeration names in the object, or a supporting document will list the enumeration items. The absence of the names in the enumeration diagram object does not mean it is an empty enumeration; the supporting document will provide the full details. If a class has several uses of the same Enumeration Type, each "line" in the diagram will include an attribute name. If an Array uses an Enumeration Type the "line" in the diagram will include the attribute name and the array notation – see Section 7.1.1.3 for information on Array Types.

Figure 4 shows a generalized example illustrating a Sensor class referencing an enumeration with the name LensSet.

[1] UInt renames the UML's UnlimitedNatural data type.

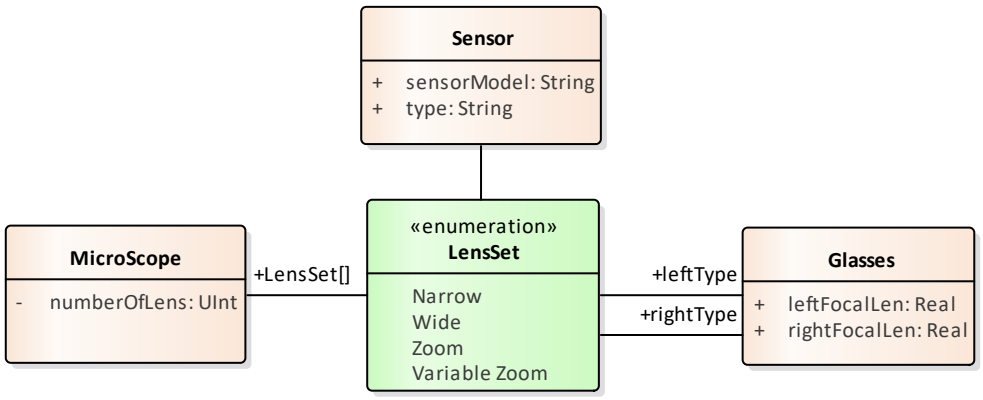


Figure 4: Enumeration Type Example

The Sensor class has only one use of the LensSet Enumeration Type so the attribute name in the Sensor class defaults to LensSet. The MicroScope class has an Array of LensSet Enumeration Types, so the diagram includes the Array Type notation (i.e., LensSet[]); the notation could use a different attribute name. The Glasses class uses the same Enumeration Type twice, so the diagram includes the attribute names for each use.

Table 2 lists the four example enumeration items for the LenSet Enumeration.

Table 2: Example Enumeration LensSet

Identifier	Name	Description
0	Narrow	A *Narrow* lens is a fixed focal length lens zoomed into the scene
1	Wide	A *Wide* lens is a fixed focal length lens zoomed out from the scene
2	Zoom	A *Zoom* lens has several fixed focal length settings each with different "zoom" levels
3	Variable_Zoom	A *Variable Zoom* lens is a continuous zoom lens

7.1.1.3 Array Type

The *Array Type* is a list of Singular Types, Enumeration Types, or References arranged as a multi-dimensional array. Arrays of numerical types (i.e., Real, Integer, and UInt) share the same minimum, maximum, and default precision trait values, if applicable. The *Array Type* notation specifies the type for each Singular Type element within the array followed by multiple dimension declarations. There are three methods for defining dimensions: Defined Length, Maximum Length, and Open Length.

A Defined Length dimension declaration starts with an open square bracket "[", a count of the number of elements, then a closed square bracket "]" to complete the definition. The element count defines the exact number of elements the attribute must support. For example, "Integer[10]" indicates an array of ten Integer values.

A Maximum Length dimension declaration starts with an open square bracket "[" a less than or equal to sign, "<=", a count of the maximum number of elements, then a closed square bracket

"]" to complete the definition. The element count defines the maximum number of elements the attribute supports, data producers define the actual number at run-time. For example, "Integer[<=10]" indicates an array containing a maximum of ten Integer values.

In an Open Length Array, the count of elements is determined at run-time, so the element count is blank. For example, "Integer[]" means the array size is determined at run-time.

Each array may have more than one dimension, for example Integer[10][][3] is an array with three dimensions: the first array dimension has ten elements (defined in the model); the second dimension is determined at run-time; and the third dimension (defined in the model) has three elements. If at run-time the second dimension becomes five, then the array would have a total of 10 * 5 * 3 = 150 elements.

Requirement(s)	
ST 1901-01	Where a producer uses an Array Type attribute which specifies a Defined Length element count for a dimension, the producer shall include the specified number of elements for that dimension in the array.
ST 1901.1-06	Where a producer uses an Array Type attribute which specifies a Maximum Length element count for a dimension, the producer shall not exceed the specified number of elements for that dimension in the array.

7.1.1.4 Tuple Type

The Tuple Type is typically a short sequence of Unsigned Integers (UInts) for use within the MIMD Model's addressing method (for Directed Association) along with other uses. The Tuple Type is the same as a one-dimensional array of UInt, but because of its short length and the restriction to one dimension, it is a separate type to enable more efficient transmutations.

7.1.1.5 RESERVED Type

The RESERVED Type is a placeholder within the attribute list for a planned future attribute. The attribute's Name and Type traits are not known when reserving the attribute; however, the attributes Identifier trait (see Section 8.1.1) is known.

7.2 MIMD Model Relationships

There are three types of MIMD Model relationships: **Composition** ("has-a" dependency relationship), **Inheritance** ("is-a" dependency relationship) and **Directed Association**. The MIMD Model diagrams show all three relationships using standard UML diagram notation.

7.2.1 Composition Relationships

The **Composition** relationship defines a "has-a" connection between a parent and a child class. The parent class "includes" the child class in a similar fashion as a class attribute, i.e., the parent has-a child. The Composition relationship can be singular or include multiplicity (i.e., multiple copies of the child class or a collection of child classes). While a child can define multiplicity, the parent cannot have multiplicities in the MIMD Composition relationship. Section 7.2.1.1 discusses Composition relationship multiplicity.

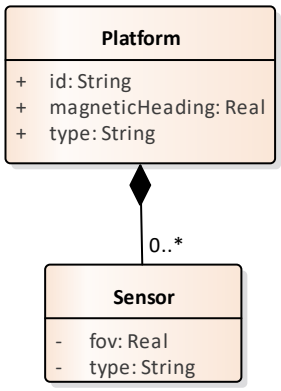


Figure 5: Composition Relationship Example

Figure 5 shows an example UML diagram with two classes (Platform and Sensor) and a Composition relationship between them. The Platform class contains three attributes: id, magneticHeading, and type, each with their associated data types of String, Real, and String, respectively. The Platform class has a fourth attribute -- the Sensor class Composition relationship, shown as a line connecting the two classes together. The filled-in diamond denotes the Platform class is the relationship parent and the Sensor class is the relationship child. Interpret the relationship as "The Platform has-a Sensor." This Composition relationship has a defined multiplicity of 0..*, meaning the Platform has zero or more Sensors (see Section 7.2.1.1). The multiplicity of the Composition relationship means the Platform's attribute of Sensor is a collection of Sensors.

The MIMD Modeling rules do not allow circular (or cyclic) Composition relationship dependency paths in the model; that is, where one class depends on one or more classes that lead back to depending on the first class. For example, a prohibited cycle of three classes is: class A depends on class B, class B depends on class C, and class C depends on class A.

7.2.1.1 Composition Relationship Multiplicity

Composite Relationship Multiplicity enables a parent class to "have-a" collection of child classes. A range of numbers on the composite relationship link defines the multiplicity (e.g., "0..5", or "1..*"). The first number is the required minimum number of elements in the collection and the second number is the required maximum number of elements in the collection. A minimum value of zero indicates the collection is optional and a minimum value greater than zero indicates a required collection when creating an instance of the parent class. A star (i.e., *) for the maximum value indicates the collection does not have an upper bound.

When the diagram does not show any multiplicity, the default multiplicity is "0..1", meaning the collection (or instance of the child) is optional, and producers may only include one element.

7.2.1.1.1 List Organization

A Composition relationship with multiplicity means the parent class has a collection, or list, of child classes. List organization is important when correlating two lists between two MIMD packets, especially when performing Report-on-Change. For example, in one MIMD packet a Platform class may have a list of elements containing sensor 1 and sensor 2. In a subsequent packet, receivers need to determine which elements of the list refer to sensor 1 and sensor 2 to correlate with the first packet.

When using Report-on-Change with lists, the producer may decide to skip sending one or more list elements in a MIMD packet to conserve bandwidth. For example, a producer may have a parent class instance containing a list of ten child instances, but the producer only sends half the child instances in a MIMD packet because only five child instances have changed since the last packet.

A **full-list** is a producer's list of child instances, i.e., the producer's in-memory list. A **packet-list** is a list of child instances within a packet. With Report-on-change, the packet-list will either be a subset of, or equal to, the full-list.

Producers have three full-list operations: addition, insertion, or deletion of one or more list elements. Replacement is the combination of an element deletion at a given index and the insertion of a new element into the same index.

Figure 6 illustrates an example of Producer Software storing a Full-List and four MIMD packets each with a Packet-List. The producer software's memory contains a MIMD structure with a platform containing a sensor full-list starting with two sensor list elements. The figure shows the Producer Software at four different times with changes to the full-list at each time.

- At Time 1, the producer creates MIMD Packet 1 with Sensor 1 and 2 in the packet-list, which illustrates the packet-list is equal to the full-list.
- At Time 2, the producer adds Sensor 3 (orange) into the full-list and when constructing MIMD Packet 2, the producer determines there is no change to Sensor 1 (i.e., static), so the producer creates a packet-list with only Sensor 2 and 3. This illustrates the "addition" operation and the producer's option to create a packet-list which is a subset of the full-list.
- At Time 3, the producer "inserts" Sensor 2a (orange) into the full-list and constructs MIMD Packet 3 with all four sensor elements in the packet-list.
- At Time 4, the producer removes Sensors 1 and 2 from the full-list and when constructing MIMD Packet 4 the producer determines there is no change to Sensor 2a (i.e., static) so, the producer creates the packet-list with Sensors 1 & 2 marked for removal and includes Sensor 3. The MIMD Transmutation Instructions (e.g., MISB ST 1902) defines the method to mark elements for removal.

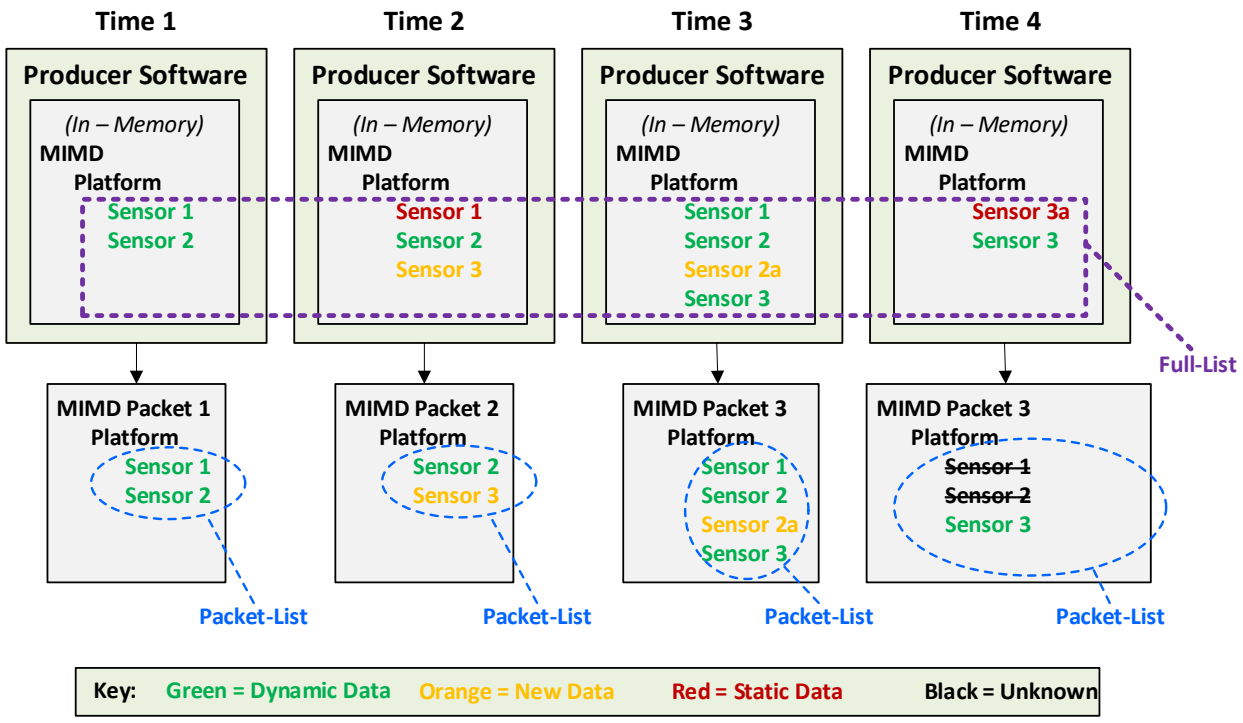


Figure 6: Lists and List Operations

This document defines full-list, packet-list, and other lists below as a method to (a) explain the behavior a producer needs to provide when using these list organization methods and (b) define list behavior requirements. This standard does not recommend, expect, or enforce any particular implementation of the list, but this standard does require the resulting behavior when sending a series of MIMD packets.

The MIMD Model supports two techniques for producers to organize packet-lists: **Fixed Order** and **Marked Element**.

7.2.1.1.1.1 Fixed Order List Organization

In a **Fixed Order** list, the position of each list element in the full-list and packet-list remains constant during the entire stream, i.e., the order of the list elements does not change (i.e., fixed) once a producer adds an element to the fill-list. Receivers use the list position to correlate list elements from packet to packet. For the Report-on-Change method, producers maintain the Fixed Order list's integrity by ensuring each list element's index is the same for every packet.

Figure 7 illustrates the list correlation in two separate packets when using Fixed Order.

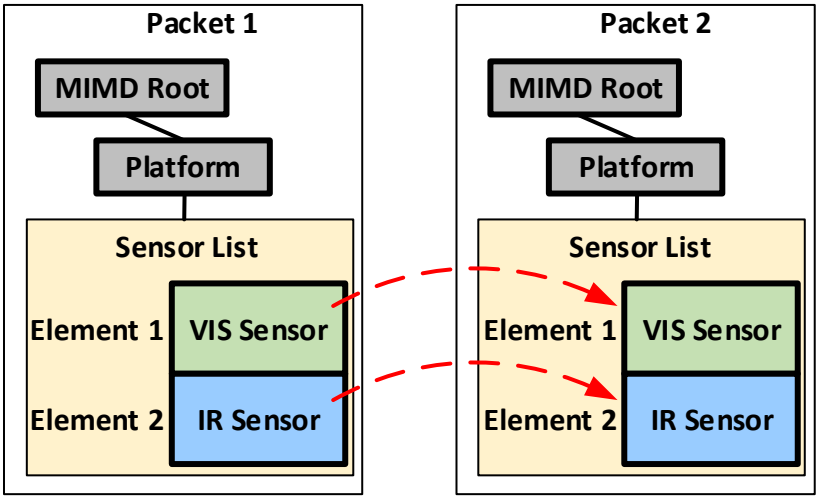


Figure 7: Fixed Order List Element Correlation

In this example the elements correlate using their position in the list; that is, receivers always know Element 1 will be VIS Sensor and Element 2 will be the IR Sensor, for the entire MIMD Stream.

A fixed-full-list is a full-list which adheres to the following rules:

1) Producers only add elements to the end of the list,
2) Producers do not remove elements from the list,
    a. Producers can pseudo-remove elements by marking the element for removal,
3) Producers do not replace elements in the list.

A fixed-packet-list is a packet-list which a producer constructs from a fixed-full-list. To support Report-on-Change, a producer uses a "filler" element to indicate a list element has not changed since the last packet. The "filler" elements maintain the element's position in the fixed-packet-list. The exception to using a filler element is when unchanged elements are at the end of the fixed-packet-list, in this case producers may drop the end elements from the fixed-packet-list to save bandwidth. The MIMD Transmutation Instructions (e.g., ST 1902) defines the method of providing filler elements.

For example, if a list has ten elements (e.g., 10 sensors) and only element seven changes (e.g., sensor 7 becomes inactive), then all elements preceding element seven (e.g., sensors 1-6) need to supply "filler" elements as placeholders in the list and all elements after seven (e.g., 8 – 10) are optional or have filler elements.

Producers "pseudo-remove" an element in the fixed-full-list by marking the element for "removal". Readers interpret the state of the removed element as ROC_Unknown. For successive packets, in the fixed-packet-list, the producer uses a "filler" element for the removed element, until either the producer chooses to re-instate the element with a value or to the end of the stream.

The advantage of a Fixed Order over a Marked Element list is fewer bytes per list element, therefore less bandwidth by not requiring the mimdId.

Requirement(s)	
ST 1901.3-10	When the producer creates a fixed-packet-list from a fixed-full-list, the producer shall ensure the position of each item in the fixed-packet-list is equal to the position of that element in the fixed-full-list.
ST 1901.3-11	Where the producer uses a fixed-full-list, the producer shall ensure the index of each item in the fixed-packet-list remains the same in every MIMD packet for the entire MIMD stream.
ST 1901.3-12	When a producer marks a fixed-full-list element for removal, while the producer does not have a value for the element, the producer shall set the corresponding fixed-packet-list element to the filler value, for all successive packets.

7.2.1.1.1.2 Marked Element List Organization

In a **Marked Element** list, each element in the full-list defines a unique identifier (i.e., MIMD Instance Identifier or mimdId) and each element in the packet-list includes its mimdId. Receivers use the mimdId to correlate elements from packet to packet. The mimdId is an attribute within the set of MIMD Base Attributes (see ST 1904) in every MIMD class.

Figure 8 illustrates correlating the list elements for two separate packets when using a Marked Element list. In this example the elements correlate using the mimdId; that is, receivers always know mimdId=13 is for the VIS Sensor and mimdId=17 is for the IR Sensor. This example shows element reversal in the second packet to demonstrate order of elements is irrelevant when using a Marked Element list.

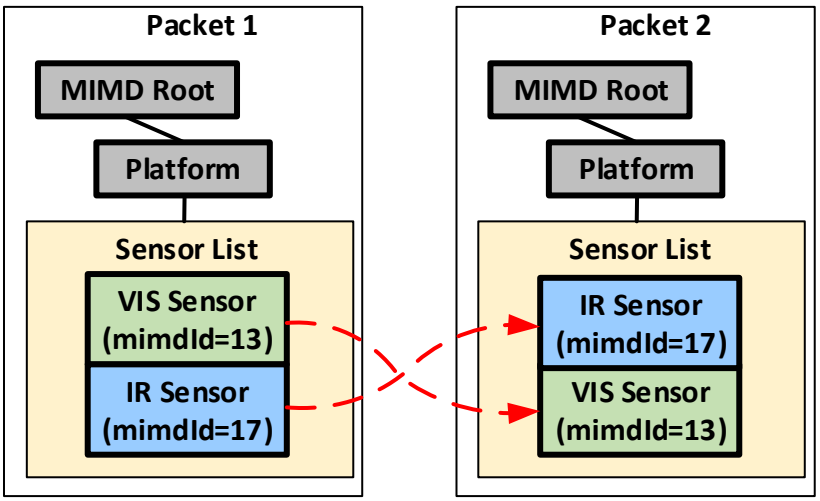


Figure 8: Marked Element List Correlation

A marked-full-list is a full-list with each element containing a mimdId. A marked-packet-list is a packet-list which a producer constructs from a marked-full-list and each element contains its mimdId.

With a marked-full-list, there are no restrictions on the list operations, because the element identifier is independent of the element's list position:

- Producers can add new elements to the end of a marked-full-list,
- Producers can insert new elements in a marked-full-list,

- Producers can remove elements from a marked-full-list,
- Producers can replace elements in a marked-full-list,
- Producers can reorder elements in the marked-full-list.

From packet to packet, the receiver will match the element's identifier to its internal list of elements to determine which element's values to update.

The advantage of a Marked Element list is random element order and the reporting of only changing elements (i.e., no placeholders). Producers can add new elements and insert elements anywhere in the list.

Requirement	
ST 1901-02	When a producer creates a marked-packet-list, each element of the list shall include its MIMD Identifier (mimdId).

7.2.1.1.1.3 List Organization Specification

When initiating a list, a producer determines whether to use either a Fixed Order or Marked Element list for the duration of the MIMD Stream. The MIMD Transmutation Instructions (e.g., ST 1902) defines the method of signaling, at run-time, which list organization method to use.

Requirement(s)	
ST 1901.3-13	Where the producer uses a Fixed Order list for a composite relationship with multiplicity, the producer shall maintain the Fixed Order list for the entire MIMD Stream.
ST 1901.3-14	Where the producer uses a Marked Element list for a composite relationship with multiplicity, the producer shall maintain the Marked Element list for the entire MIMD Stream.

7.2.2 Inheritance Relationship

The **Inheritance** relationship defines an "is-a" connection between a parent and a child class. The child inherits all the attributes of the parent, so the child "is-a" new extension, or specialization, of the parent class. In effect, the child class is a more specific version of the more general parent class. The MIMD Model does not allow Redefinition (as described in the UML specification) of the MIMD Base Attributes. A child class does not define its own MIMD Base Attributes, instead the child's Base Attributes are the same as the parent's Base Attributes. Neither the child nor parent define multiplicities with an Inheritance relationship.

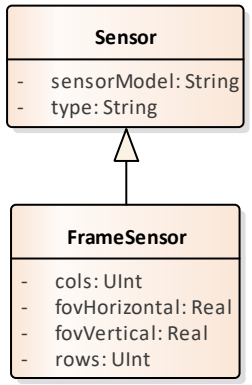


Figure 9: Inheritance Relationship Example

Figure 9 shows an example of an Inheritance relationship. The Sensor class contains two attributes, the sensorModel and type. The Sensor class is a generalization or superclass of the FrameSensor class, where the FrameSensor class includes, or inherits, all attributes of the Sensor class. A FrameSensor has six attributes: cols, fovHorizontal, fovVertical, and rows – plus the Sensor class attributes – sensorModel and type. Interpret the relationship as "FrameSensor is-a Sensor."

The MIMD Modeling rules do not allow circular (or cyclic) Inheritance relationship dependency paths in the model. The MIMD Modeling rules do not allow multiple inheritances; therefore, child classes have Inheritance Relationships with only one parent.

Inheritance relationship parent classes may have several children classes providing different specializations. When a producer instantiates a child instance in an Inheritance relationship, the child cannot switch to a different specialization for the entirety of the MIMD Stream (i.e., a parent class instance only allows one child instance).

Requirement(s)	
ST 1901.3-15	Where the producer instantiates a specialization class (child) instance of a parent class instance, the child instance's class type shall remain constant for the entire MIMD Stream.
ST 1901.3-16	Where the producer instantiates a specialization class (child) instance of a parent class instance, the child instance shall be the only specialization instance the parent instance relates to for the entire MIMD Stream.

7.2.3 Directed Association Relationship (Class to Class Reference)

The MIMD Model uses **Directed Association** relationships to define a "refers-to" link between a referring instance and destination instance. The referring instance has an attribute that "points" to the destination instance using the destination's unique instance identifier (i.e., mimdId), which is part of the MIMD Base Attributes. The Directed Association relationship does not provide any data containment, such as Composition or Inheritance; it provides only a linkage between class instances.

Figure 10 shows an example of Directed Association from the Encoder class (referring class) to the Sensor class (destination class). The diagram shows a Platform with composition for Sensor(s), and Encoder(s). The Sensor class includes the MIMD Base Attributes, one of which is a unique instance identifier - mimdId (see Section 7.2.3.1). The Encoder includes an attribute, called "source" that contains the Sensor instance's mimdId value. The Base Attributes are not visible in the MIMD class diagrams.

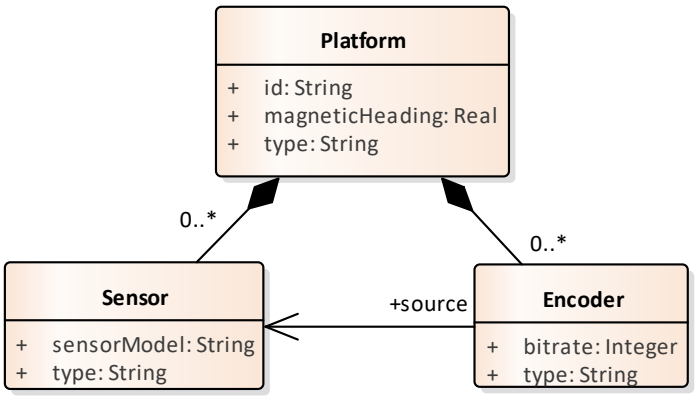


Figure 10: Example classes using Directed Association

During normal system operation, a Sensor passes data to one or more Encoders. Which Sensor connects to which Encoder can change at any time. Within this example the Encoder class is not

a composition of the Sensor class; thus, the Directed Association shows which Encoder connects to which Sensor. By having each Sensor class instance define a mimdId, and each Encoder class includes a reference attribute (i.e., source), an Encoder can "link" to a Sensor from which it receives input.

Figure 11 illustrates two pairs of Sensor and Encoder class instances and shows the references for each encoder's source input in red. Each Sensor class instance defines a mimdId, shown in parenthesis, and each Encoder class instance shows what the *source* attribute links to. In the figure, Encoder 1 receives its input from Sensor 1 and Encoder 2 receives its input from Sensor 2. At run-time, if a system needs to change which encoder links to which sensor, the value of *source* changes appropriately.

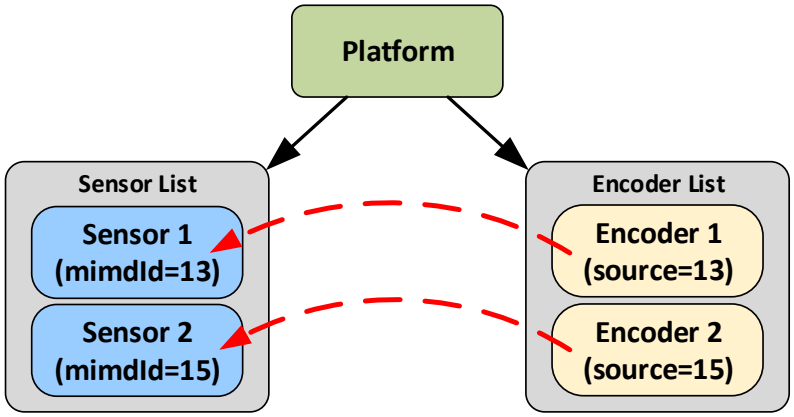


Figure 11: Example Soft Links Between Instances

7.2.3.1 Instance Identifier – mimdId

The mimdId is the class instance's unique identifier, which is a two value Tuple: *serial number* and *identifier group.* The instance serial number is a unique unsigned integer value assigned at run time by a data creator. The identifier group defines a group to which an instance identifier belongs. Systems generating the data assign group identifiers, as needed, to sub-components of the system. Group zero is the default group if the group identifier is not included in the Tuple (some transmutations may remove default group identifiers to save bandwidth).

The instance serial number is unique within the scope of the group so a system using multiple group identifiers may have instances with the same serial number but different group numbers. The combination of instance serial numbers with the group identifier creates a globally unique identifier for every instance in the MIMD model.

MISB ST 1904 provides details on the use of the mimdId within the MIMD model.

7.2.3.2 Arrays of Directed Association Relationships

The MIMD Model allows arrays of Directed Association relationships. This enables a class to define multiple references to the same class multiple times, thereby having a list of instance references. Figure 12 illustrates a Sensor class with a PointOfInterest class list and a PhysicalObject class which references a subset of the points of interest using an array of directed associations.

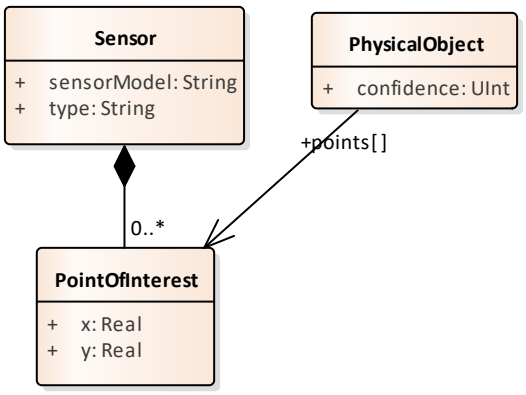


Figure 12: Example Directed Association Array (Reference Array)

7.2.4 MIMD Hierarchy

The MIMD model, MISB standards ST 1903 thru ST 1908, defines the class hierarchy which applications use to build instance hierarchies. The MIMD hierarchy starts with a singular MIMD class instance as the root and producers set attribute values and create child class instances per the MIMD class relationship definitions. The child instances may have one or more of their own children, "grandchildren", great grandchildren, etc. class instances making MIMD hierarchical.

Figure 13 shows a Class Diagram example on the left and an Instance Diagram example on the right. The class diagram shows the MIMD class with a version attribute and the class has a composition relationship with the platform class (i.e., the MIMD class "has a" platform class). The platform class defines attributes (i.e., id, magneticHeading, type) and "has a" sensor class with multiplicity and the platform class "has a" stage class with multiplicity. The sensor class defines attributes and "has a" stage class with multiplicity. The stage class has position and orientation attributes. The instance diagram illustrates what a software application builds to store the information for a particular application need. The instance diagram defines a MIMD instance with the "version" attribute and the MIMD instance contains a Platform instance. In this example, the application only needs the Platform class's "id" attribute. The Platform instance contains a list of Stage instances and a Sensor instance. The Stage instances contain position and orientation data about the Platform; therefore, these are the Platform's Stage instances. The Sensor instance defines both the "type" and "sensorModel" attributes and contains a list of Stage instances. The Stage instances contain position and orientation data about the Sensor; therefore, these are the Sensor's Stage instances. The Platform and Sensor Stage instances are unique and relate directly to their parent, either the Platform or the Sensor.

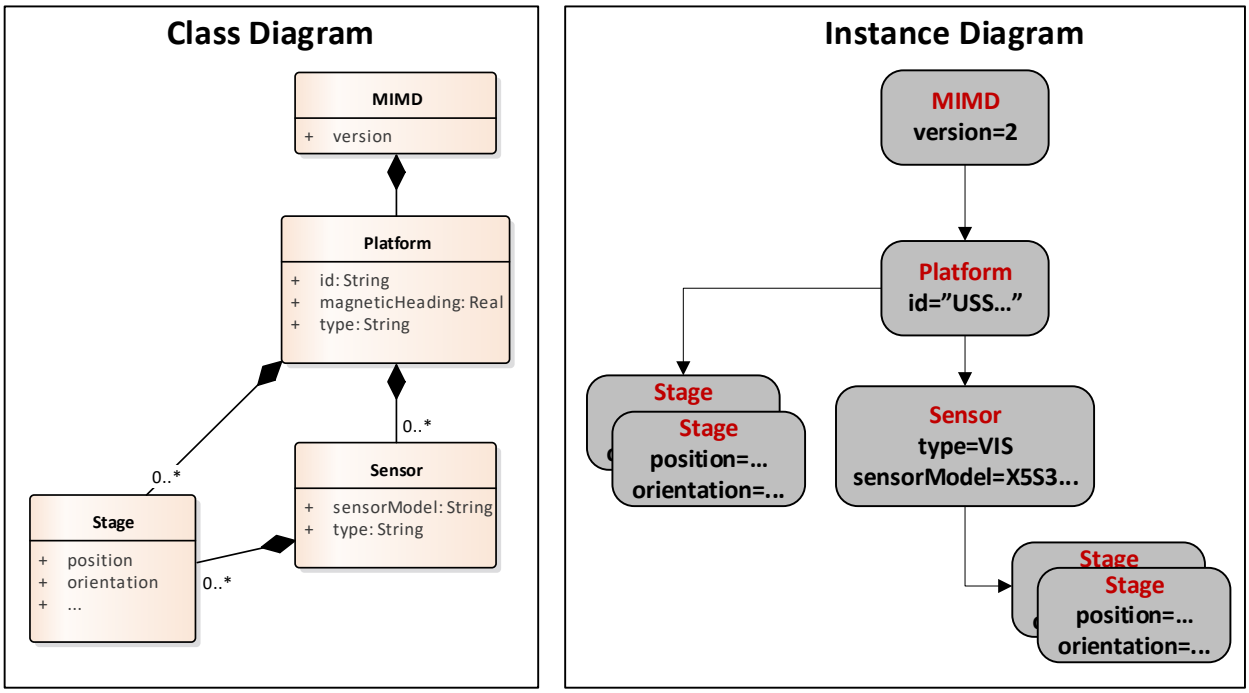


Figure 13: Hierarchy Example

The example illustrates a type of "ownership" a parent class (i.e., Platform or Sensor) has with its children's classes (i.e., Stage). Where both the Platform and Sensor use the same class in their definitions, the Platform "owns" its Stage instances, and the Sensor owns its Stage instances. This "ownership" concept applies to all classes in the hierarchy.

Except for the root (i.e., MIMD) instance, all classes are a part of the context of their parent class, therefore, every instance in the hierarchy can define a "path" to the root node.

With Report-on-Change, when an instance attribute value changes, and a producer constructs the MIMD packet, the packet must include the structure of the parent(s) to the changed value.

Requirement(s)	
ST 1901.3-17	Where the producer includes an instance in MIMD packet, the MIMD packet shall include the instance's hierarchical parent instances, up to and including the MIMD class.

7.3 MIMD Model Diagram External References

The complete MIMD Model diagram is large, so a collection of documents (i.e., MISB ST 1903, ST 1904, etc.) divides the diagram into logical groups and provides the details. When a document needs to refer to a class in a different group (e.g., within the current document or another document) the class will have the name of the document, two colons ("::") and the name of the class; furthermore, the class will have a yellow highlight. Figure 14 illustrates an example of an external reference where the Platform class refers to the Stage class. This example shows the Stage class in yellow. All the Stage class's children and references are not visible; however,

as indicated with the "ST1906" designation, the Stage class's full diagram, attributes, and details (including its children and references) are available in MISB ST 1906.

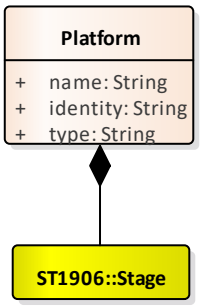


Figure 14: External Reference Example

8 MIMD Textual Class Definitions

The MIMD UML Modeling Rules in Section 7 define a graphical depiction of the MIMD Model. The MIMD Textual Class Definitions define the formal written description of the MIMD Model. The textual model enables transitioning the UML model into software and aids encoding the MIMD Model with different formats (e.g., KLV or XML). Each MIMD standard defining a MIMD class includes a table definition for the class. The table provides relationship and attribute details including all the traits and a full description of each attribute's purpose. The table definitions contain the normative information for each class. The normative class name is in the table caption and the normative attribute names are in the table along with the attribute's characteristics in the table columns. Section 8.4 describes the table columns and provides an example of a graphical class model and its table definition.

The MIMD Textual Class Definitions provide the rules and requirements for detailing information about each class and its attributes. These rules and requirements build upon the MIMD UML Modeling Rules in Section 7. The textual rules and requirements include additional attribute traits and defines class relationships and enumerations as attributes with special types.

8.1 Additional Attribute Traits

Section 7.1.1 defines the basic attribute traits (i.e., Name and Type). The MIMD Textual Class Definitions define an additional required trait (i.e., Identifier) and other optional traits depending on the type of attribute.

8.1.1 Identifier Trait

The attribute trait **Identifier** is a required trait; it is an unsigned integer unique within the scope of the class. The MISB assigns permanent attribute Identifiers which ensures backwards compatibility with future updates to the model. Since an Identifier is unique only within the class scope, two different classes may have an attribute with the same Identifier. For example, a Platform class has a magnetic heading (magneticHeading) attribute with an Identifier equal to 33, while a Sensor class has a field-of-view (fov) attribute with an Identifier equal to 33. However,

within the *scope* of the class the Identifier of 33 is unique across all other Identifiers. During KLV Transmutation (see MISB ST 1902) the attribute Identifier becomes a Local Set tag.

8.1.2 Optional Traits

The optional traits are: **Minimum**, **Maximum**, **Maximum Length**, **Units**, and **Deprecate**. Optional traits apply to the attribute under certain conditions; for example, with an attribute type Real the Minimum trait may apply.

8.1.2.1 Minimum and Maximum

The attribute traits **Minimum** and **Maximum** provide the numeric bounds for an attribute of numeric type (Real, Integer, or UInt) and define the limits of Composite Relationship multiplicities. For numeric types both the Minimum and Maximum traits are values with the Minimum value less than the Maximum value. These two traits enable range checking and validation. A model document may state the Minimum and Maximum trait values as numeric values or use the constants in Table 6 of Appendix A. If the attribute does not define either the Minimum or Maximum trait, the default numeric range is -infinity to +infinity, i.e., $(-\infty, +\infty)$. If the attribute only defines the Minimum trait the inclusive numeric range is from the minimum value to +infinity, i.e., $[min_{value}, +\infty)$. If the attribute only defines a Maximum value, the numeric range is from -infinity to the inclusive maximum value, i.e., $(-\infty, \max_{\text{value}}]$.

The Minimum and Maximum trait numeric values follow the International System of Units (SI Units) [13] method for "Formatting numbers, and the decimal marker" by using a space for digit grouping and a period for the decimal marker.

For Composite Relationship multiplicities the Minimum and Maximum trait values define the required minimum and maximum number of elements in the collection.

Requirement	
ST 1901.2-07	Where a numeric attribute specifies the Minimum trait, the instance value shall be greater to or equal to the Minimum trait value.
ST 1901.2-08	Where a numeric attribute specifies the Maximum trait, the instance value shall be less than or equal to the Maximum trait value.

8.1.2.2 Maximum Length

The attribute trait **Maximum Length** states the maximum number of elements a value can contain. For example, a String value may have a limited length imposed by the model. The Maximum Length applies to the String and Tuple types. For strings the Maximum Length is the maximum number of characters (e.g., UTF-8 characters). For Tuples, the Maximum Length is the number of UInts. The Maximum Length is **not** a maximum number of bytes.

Requirement	
ST 1901-04	Where an attribute specifies the Maximum Length trait, the producer shall limit the instance value length to the number of elements (e.g., characters) the Maximum Length defines.

8.1.2.3 Units

The attribute trait **Units** provides the unit of measurement for the quantity indicated. Models will specify all Units in International System of Units (SI Units). This standard prohibits Model updates from creating attributes with the same meaning as another attribute but with different units or measurement scales.

8.1.2.4 Deprecate

The attribute trait **Deprecate** denotes the attribute is invalid for use (i.e., deprecated). Section 10 details how and when to apply this trait.

8.2 Relational and Enumeration Attributes

The MIMD UML diagrams list relational and enumeration attributes as lines (e.g., composite relationships) between classes. The MIMD Textual Class Definitions are a written (non-graphical) specification of the model, so a class includes the relationships and enumeration types as attributes. The following sections define the textual representation of the three types of MIMD relationships (from Section 7.2) and the Enumeration Type (from Section 7.1.1.2); that is, Composite Relationship, Inheritance Relationship, Directed Association, and Enumeration Type.

8.2.1 Composite Relationship

The Composite Relationship is a "has-a" relationship where a parent class "has-a" child class. The MIMD system treats this relationship as containment, where the parent contains the child classes attributes within a single attribute of the parent classes attributes. Composition relationships form the model hierarchy and may refer to a single class or a collection of the same class. The Composite Relationship defines the containment of the child class within the context of the parent class, so when encoding the child class's information (in any format such as KLV or XML) the child's attribute data embeds within the parent's class data in some form which is decodable back into a separate child class. Section 7.2.1 discusses the graphical model for Composition Relationships.

The parent class of a Composite Relationship textually represents a child class as another attribute in the parent class. Each singular Composition relationship defines the Name, Type, Identifier, and optionally Deprecate traits. By default, the Name trait is set to the child class's name but with the first letter in lowercase to comply with the attribute naming rules. The Type trait for a singular Composition relationship is also the child class's name. The Type trait for a Composition relationship with multiplicities is "LIST <*x*>", where *x* is the child class's name. The Identifier trait is an unsigned integer unique within the scope of the parent class's attribute list. If a parent class needs two or more Composition relationships to the same child class, each relationship will have unique Name and Identifier traits. The Minimum and Maximum trait

values define the number elements in a Composite Relationship multiplicity. Where there is no Composite Relationship multiplicity the Minimum and Maximum traits are set to "--" (double dash).

Section 8.4 includes an example of a composite relationship in the class table. The row with Id 41 is an example of a single composite relationship attribute, and row 42 is an example of a Composite Relationship attribute with multiplicities.

8.2.2 Inheritance Relationship

The Inheritance Relationship is a merging of parent and child attributes into a single "combined" class. Ideally, the "merging" would form a new class with all the attributes from the parent and child in one class; however, merging attribute traits (specifically the Name and Identifier traits) causes a modeling data-management issue. To rectify the data-management issue the parent class contains the child class the same way as the Composite Relationship does, i.e., as an attribute.

A parent class may have multiple inheriting children; however, during implementation, inheritance rules allow only one inheritance relationship per parent when instantiating a class. For example, a Position class may have an inheriting child class for Geodetic position and another inheriting child class for Geocentric position. When instantiating a Geodetic [and therefore the Position] class, the Position class only contains the Geodetic attribute; the UML inheritance rules forbid including the Geocentric attribute in the Position class. Section 7.2.2 discusses the graphical model for inheritance relationships.

Each Inheritance relationship defines an attribute with Name, Type, Identifier, and optionally Deprecate traits. By default, the Name trait is set to the child class's name but with the first letter in lowercase to comply with the attribute naming rules. The Type trait is the child's class name. The Identifier trait is a unique number within the scope of the parent classes attribute list. The Identifiers for the child class attributes are independent of the parent class, so they start at the value of 33.

Section 8.4 includes an example of an inheritance relationship in the class table. The row with Id 46 is an example of an inheritance relationship attribute.

8.2.3 Directed Association (Class to Class Reference)

The Directed Association is a reference between a referring instance and destination instance within the model. The reference only identifies an instance in the hierarchy, i.e., it is a "pointer" to an instance. The referring class textually represents the Directed Association as an attribute in its class. Unlike the Composite Relationship Type, the destination instance does not embed within the referring class; only the address (i.e., mimdId) of the destination instance is in the referring class. Section 7.2.3 discusses the Directed Associations.

Each Directed Association relationship defines the Name, Type, Identifier, and optionally Deprecate traits. The Name trait is a unique name within the classes attribute list. The Type trait is "REF<x>", where x is the destination class's name, e.g., the type of a reference to a "Sensor" class is "REF<Sensor>". The Identifier trait is a unique number within the scope of the parent classes attribute list.

Section 8.4 includes an example of a directed association in the class table. The row with Id 44 is an example of a directed association attribute.

Arrays of Directed Association use the bracket notation after the "REF<x>" to denote an array of references, e.g., "REF<Sensor>[]" indicates an array of references to Sensor class instances.

8.2.4 Enumeration Type

An Enumeration Type is a list of Enumeration Items one of which assigns to an attribute's value. The class using an Enumeration Type textually represents the enumeration use as another attribute in the class. Section 7.1.1.2 defines the details of each Enumeration Item.

Each class which refers to an Enumeration Type defines an attribute with Name, Type, Identifier, and optionally Deprecate traits. By default, the Name trait is set to the Enumeration Type's name but with the first letter in lowercase to comply with the attribute naming rules. If a single class uses the same Enumeration Type multiple times, each attribute in the class will have a unique name and identifier. The Type trait is the Enumeration Type's name. The Identifier trait is a unique number within the scope of the parent classes attribute list.

Section 8.4 includes an example of an Enumeration Type in the class table. The row with Id 43 is an example of an Enumeration Type attribute.

8.3 MIMD Model Document Structure

MIMD Model documents follow the similar structure of all MISB document format. MIMD Model documents have two required top-level sections:

- **Model Classes** – Serial list of all classes, with their attributes, and one or more UML diagrams visually representing the classes. Each class defines an Attribute Table; Section 8.4 defines the common table structure.
- **Model Enumerations** – List of all enumeration objects, with their list of individual items including the identifier, name, and a brief description. Section 8.5 provides an example of the Enumeration section.

8.4 Attribute Table Format

All class definitions will include an attribute table which lists all the class's attributes (except the base attributes), and their traits. The table has nine columns of trait and descriptive information: **Id**, **Name**, **Type**, **Min**, **Max**, **MLen**, **Units**, and **Ref**.

- The **Id** trait is the unique numerical identifier for the attribute; that is, unique within the scope of the class. The Id is an unsigned integer starting at a value of 33 for all class descriptions (the Base Attributes reserve Id's 1 through 32).
- The **Name** trait is a unique name (within the scope of the class) for the attribute. Section 7.1.1 defines the rules for the attribute Names.
- The **Type** trait defines the data representation of the attribute value once the class has been instantiated. There are four categories of Types: Singular, Array, Enumeration, and Relationship.

  - Section 7.1.1.1, Section 7.1.1.2, and Section 7.1.1.3 define Singular, Enumeration Types and Array, respectively.
  - MIMD Models use three kinds of Relationships: Composite, Inheritance, and Directed Association.
    - The Type names for Composite are the child class name. The Type names for Composite Relationship lists prepend the child class name with "LIST<" and have a suffix of ">", for example "LIST<Sensor>".
    - The Type name for Inheritance is the name of the child class.
    - The Type names for Directed Association prepend the child class name with "REF<" and have a suffix of ">", for example "REF<Sensor>".

- The **Min** and **Max** traits are for numerical Types and Composite Relationship multiplicity bounds; otherwise, they are set to either "N/A" for Not Applicable, or "--" (double dash) for unknown. For numeric types, the Min and Max are the Minimum and Maximum bounds of the numeric attribute. For Composite Relationship multiplicities, the Min and Max are the allowed number of child elements in the multiplicity collection. A min value of zero indicates the whole list may be empty (and therefore optional). A min value of greater than zero indicates the minimum required number of elements in the list. A "*" for the max represents an unlimited maximum number of elements.
- The **MLen** trait defines the Maximum Length for a String or Tuple Type; otherwise, it is set to "N/A" for Not Applicable.
- The **Units** trait defines the units of measure for the attribute or "None" if units do not apply. All units follow the SI standard.
- The **Ref** column provides a sub-section reference that is one level below the class section level (e.g., if a class's section number is 7.2, the class attributes sub-sections' will start at 7.2.1.

Table 3 summarizes the abbreviations found in a Class Attribute Table.

Table 3: Class Attribute Abbreviations

Word or Phrase	Abbreviation
Identifier	Id
Name	Name
Type	Type
Minimum	Min
Maximum	Max
Maximum Length	MLen
Units	Unit
Reference	Ref
Deprecate	Deprecate
Not Applicable	N/A
Unknown	--

Figure 15 shows a hypothetical class called ExampleClass to illustrate the diverse types of attributes and relationships. The diagram shows other classes also but without their attributes.

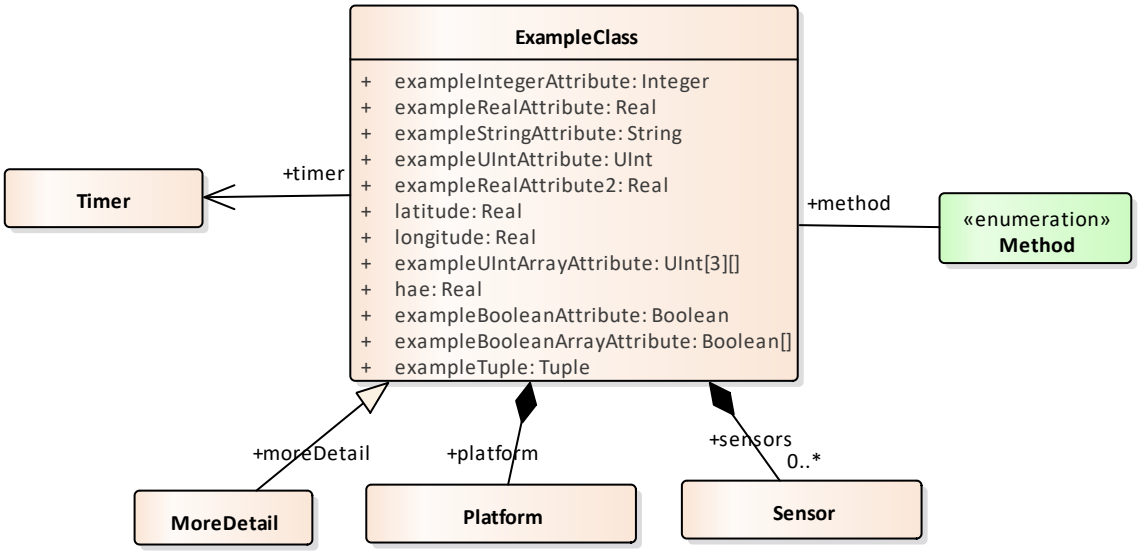


Figure 15: Example Model for Attribute Table Example

Table 4 shows an attribute table for the ExampleClass class in Figure 15.

Table 4: Example Class Attribute Table

Id	Name	Type	Min	Max	MLen	Units	Ref
33	exampleIntegerAttribute	Integer	-100	128	N/A	ns	8.4.1
34	exampleRealAttribute	Real	--	--	N/A	m	8.4.2
35	exampleStringAttribute	String	N/A	N/A	120	None	8.4.3
36	exampleUIntAttribute	UInt	0	53	N/A	None	8.4.4
37	exampleRealAttribute2	Real	0	PI	N/A	rad	8.4.5
38	latitude	Real	-PI	PI	N/A	rad	8.4.6
39	longitude	Real	-PI	PI	N/A	rad	8.4.7
40	exampleUIntArrayAttribute	UInt[3][]	--	--	N/A	None	8.4.8
41	platform	Platform	N/A	N/A	N/A	None	8.4.9
42	sensors	LIST<Sensor>	0	*	N/A	None	8.4.10
43	method	Method	N/A	N/A	N/A	None	8.4.11
44	timer	REF<Timer>	N/A	N/A	N/A	None	8.4.12
45	hae	Real	-900	20000	N/A	m	8.4.13
46	moreDetail	MoreDetail	N/A	N/A	N/A	None	8.4.14
47	exampleBooleanAttribute	Boolean	N/A	N/A	N/A	None	8.4.15
48	exampleBooleanArrayAttribute	Boolean[]	N/A	N/A	N/A	None	8.4.16
49	exampleTuple	Tuple	N/A	N/A	2	None	8.4.17

The following sub-sections provide example descriptions for the attributes in Table 4. Normally the descriptions discuss the meaning of the attribute and do not repeat information from the attribute table as these do.

8.4.1 Attribute 33 - exampleIntegerAttribute

Attribute 33 is an Integer attribute with a minimum allowed value of -100 and maximum allowed value of 128.

8.4.2 Attribute 34 - exampleRealAttribute

Attribute 34 is a Real attribute with an unlimited minimum and maximum value.

8.4.3 Attribute 35 - exampleStringAttribute

Attribute 35 is a String attribute with a maximum of 120 characters (e.g., UTF-8).

8.4.4 Attribute 36 - exampleUIntAttribute

Attribute 36 is an unsigned integer (UInt) attribute with a minimum allowed value of zero (by definition of UInt) and a maximum allowed value of 53.

8.4.5 Attribute 37 - exampleRealAttribute2

Attribute 37 is a Real attribute with a minimum allowed value of -PI (i.e., -π) and a maximum allowed value of PI (i.e., π), which is a constant defined in Appendix A.

8.4.6 Attribute 38 - latitude

This section demonstrates the use of a combined description of attributes 38, 39, and 45 (attribute sections for attributes 39 and 45 “refer” to this section). The latitude, longitude, and hae attributes are three parts of a coordinate so they share a combined description. Shared descriptions do not need to be sequential attributes in the attribute table, as shown with hae, Id=45.

8.4.7 Attribute 39 – longitude

Refer to attribute 38, latitude, for the details of this attribute.

8.4.8 Attribute 40 - exampleUIntArrayAttribute

Attribute 40 is an example of a two-dimensional array of Unsigned Integers, where the number of rows is 3 and the number of columns is determined at runtime.

8.4.9 Attribute 41 - platform

Attribute 41 is an example of a composite relationship attribute.

8.4.10 Attribute 42 - sensor

Attribute 42 is an example of an attribute that is a collection of child classes. The collection is optional since the minimum trait is zero. The collection may include an unlimited number of elements since the maximum trait value is set to star (*).

8.4.11 Attribute 43 - method

Attribute 43 is an example of an Enumeration, where Table 5 defines the *Method* Enumeration

8.4.12 Attribute 44 - timer

Attribute 44 is an example of a reference to a Timer class.

8.4.13 Attribute 45 – hae

Refer to attribute 38, latitude, for the details of this attribute.

8.4.14 Attribute 46 - moreDetail

Attribute 46 is an example of an inheritance relationship to the ExampleClass.

8.4.15 Attribute 47 - exampleBooleanAttribute

Attribute 47 is an example of a single Boolean attribute.

8.4.16 Attribute 48 - exampleBooleanArrayAttribute

Attribute 48 is an example of an array of Boolean values.

8.4.17 Attribute 49 – exampleTuple

Attribute 49 is an example of a “two” Tuple which supports two UInts.

8.5 Enumeration Section Format

Enumeration tables consist of three columns: Identifier, Name, and Description. Section 8.5.1 illustrates an enumeration section.

8.5.1 Method Enumeration

This shows an example enumeration section. See Table 5 for list of enumeration values.

Table 5: Enumeration Values for Method

Identifier	Name	Description
0	Fire	Target is burning
1	Water	Target is wet
2	Air	Target is flying

9 Motion Imagery Modeling Language

The Motion Imagery Modeling Language (MIML) is a concise textual representation of a model following the rules from Section 7 and Section 8. The model includes: the class definitions with their attributes plus their traits, the relationships between classes, and enumerations. A formal grammar which enforces the rules and structure of the model defines the syntax of MIML. Future updates of the MIMD model standards will use the MIML description as the normative form of the MIMD model.

10 Model Management

To reduce backwards compatibility issues for MIMD Model developments and revisions, all updates to attributes and classes will be additive. Model developers have three possible options for changes to class attributes and classes: Create, Delete, and Update.

1. Create - When creating a new attribute, it will have a unique Name and Identifier within the scope of its class. Similarly, all new classes will have unique names.
2. Delete - To delete an attribute, the attribute's Deprecate trait is set to the current version, meaning the attribute is not for use in any application from the named version forward. Model developers are not to reuse the attribute's Name or Identifier for any other attribute in the class. When deprecating all relationships to a class, including association attributes via the mimdId, the whole class is no longer a part of the model – i.e., it is deleted.
3. Update - Model developers will not make updates to existing attributes that change any of the attribute's traits. Instead, if an attribute needs an update, model developers *create* (see above) a new attribute and *delete* (see above) the existing attribute. Model developers will not change class names but instead create new classes and the appropriate relationship attributes.

There are two version incompatibility situations: (1) old version producer to new version receiver and (2) new version producer to old version receiver. For case (1) if a new version receiver only accepts the new version model data, the receiver will ignore all attributes marked for deprecation in the new model. Attributes common in the old and new versions will function normally.

For case (2) an old version receiver must ignore attributes it does not know about and expect some attributes to be missing (i.e., if the new version had them deleted). All attributes common in the old and new version will function normally.

Requirement	
ST 1901-05	When a receiver detects an unknown attribute, the receiver shall ignore or skip the attribute.

11 Retired Requirements

The following requirement has been retired.

Requirement	
ST 1901-03 (Retired)	Where a numeric attribute specifies the Minimum and Maximum traits, the instance value shall not exceed the numeric range defined by these bounds.

Appendix A Constants

Attribute traits may use the constants listed in Table 6.. To clearly indicate a value is a constant in the attribute tables, all constant names are CAPITALIZED, individual words are separated by underscores (e.g., CONSTANT_NAME. For irrational numbers, values in this table only specify up to 15 digits to provide enough precision for double precision IEEE values.

Table 6: MIMD Modelling Constants

Constant	Value	Description
PI	3.141592653589793	The circumference of a circle divided by its diameter. The value of PI (or π) is irrational and has an infinite number of digits after the decimal point. In Radians a value of PI equals 180 degrees, or a half circle.
HALF_PI	1.5707963267948965	One half the value of PI, i.e., $\frac{\pi}{2}$. In Radians the value of HALF_PI equals 90 degrees, or a quarter of a circle.
TWO_PI	6.283185307179586	Two times the value of PI, i.e., 2π. In Radians the value of TWO_PI equals 360 degrees, a full circle.